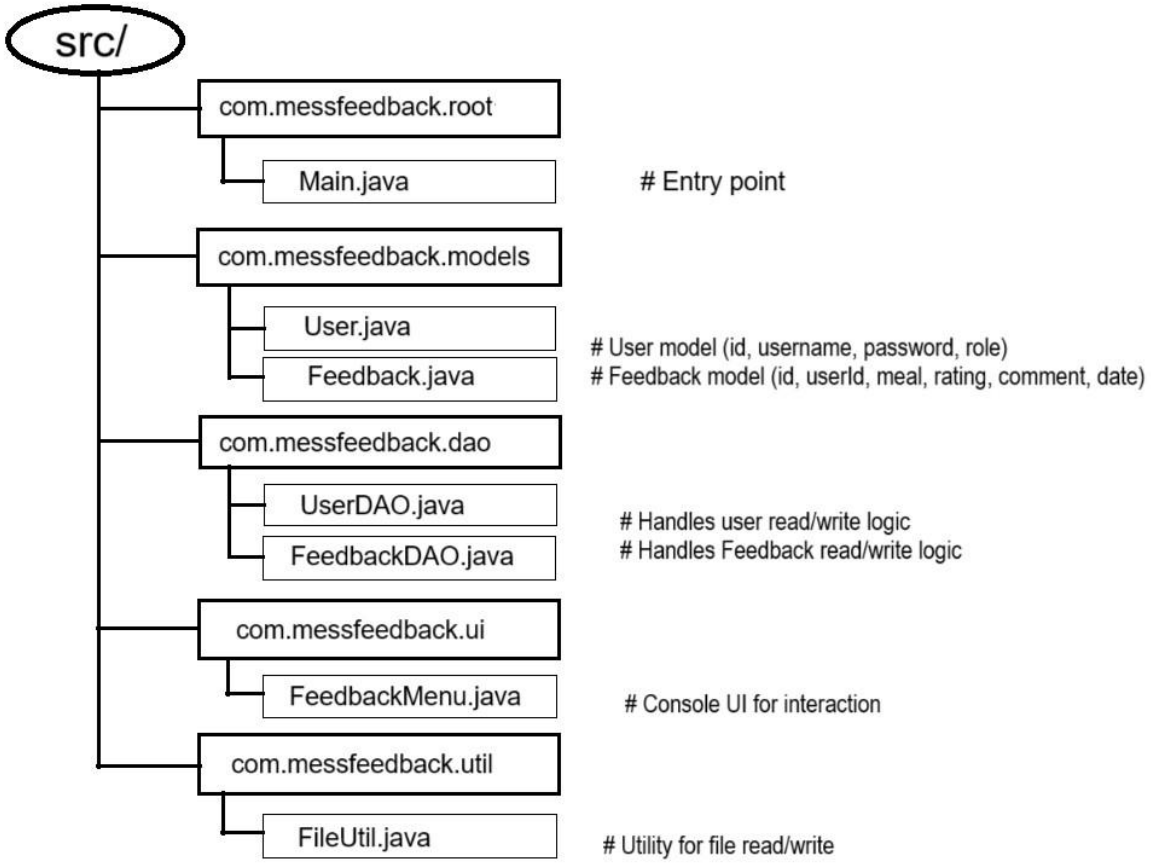
Project Structure



1. models Package

Class: Feedback

Purpose: This is the data model class used to represent feedback details submitted by a student.

Fields: - int id – Unique identifier for each feedback. - String studentName – Name of the student who gave feedback. - String mealType – Type of meal (Breakfast/Lunch/Dinner). - int rating – Rating given (typically from 1 to 5).

- String comments – Student's opinion or suggestion. - String date – Date of feedback submission.

```
package com.messfeedback.models;

/**
 * Model class representing a feedback entry.
 */
```

```
public class Feedback {
    private int feedbackId;   // Unique ID for each feedback
    private int userId;       // ID of the user who submitted the feedback
    private String mealType;  // Meal type (Breakfast, Lunch, Dinner)
    private int rating;       // Rating given by the user (1 to 5)
    private String comments;  // Optional comments about the meal
    private String date;      // Date of feedback submission (stored as String for simplicity)

    /**
     * Constructor to initialize a Feedback object.
     */
    public Feedback(int feedbackId, int userId, String mealType, int rating, String comments,
String date) {
        this.feedbackId = feedbackId;
        this.userId = userId;
        this.mealType = mealType;
        this.rating = rating;
        this.comments = comments;
        this.date = date;
    }

    // === Getter and Setter Methods ===

    public int getFeedbackId() {
        return feedbackId;
    }

    public void setFeedbackId(int feedbackId) {
        this.feedbackId = feedbackId;
    }

    public int getUserId() {
        return userId;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }
```

```
public String getMealType() {
    return mealType;
}

public void setMealType(String mealType) {
    this.mealType = mealType;
}

public int getRating() {
    return rating;
}

public void setRating(int rating) {
    this.rating = rating;
}

public String getComments() {
    return comments;
}

public void setComments(String comments) {
    this.comments = comments;
}

public String getDate() {
    return date;
}

public void setDate(String date) {
    this.date = date;
}

/**
 * Converts feedback object to a readable string format.
 */
@Override
public String toString() {
```

```
        return "Feedback{" +
                "feedbackId=" + feedbackId +
                ", userId=" + userId +
                ", mealType='" + mealType + '\'' +
                ", rating=" + rating +
                ", comments='" + comments + '\'' +
                ", date='" + date + '\'' +
                '}';
    }
}
```

class: User

```
package com.messfeedback.models;

public class User {
    private int userId;
    private String username;
    private String password;
    private String role; // "student" or "admin"

    public User(int userId, String username, String password, String role) {
        this.userId = userId;
        this.username = username;
        this.password = password;
        this.role = role;
    }

    // Getters and Setters
    public int getUserId() {
        return userId;
    }
    public void setUserId(int userId) {
        this.userId = userId;
    }
    public String getUsername() {
        return username;
    }
    public void setUsername(String username) {
```

```
        this.username = username;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
    public String getRole() {
        return role;
    }
    public void setRole(String role) {
        this.role = role;
    }

    @Override
    public String toString() {
        return userId + "," + username + "," + password + "," + role;
    }

    public static User fromString(String line) {
        String[] parts = line.split(",");
        return new User(Integer.parseInt(parts[0]), parts[1], parts[2], parts[3]);
    }
}
```

Class: Vote(Core Feature)

```
// Vote.java
package com.messfeedback.models;

public class Vote {
    private int voteId;
    private int userId;
    private int mealChoice; // 1 = Veg Biryani, 2 = Rajma Chawal, 3 = Paneer Butter Masala
    private String date;

    public Vote(int voteId, int userId, int mealChoice, String date) {
        this.voteId = voteId;
```

```
        this.userId = userId;
        this.mealChoice = mealChoice;
        this.date = date;
    }

    public int getVoteId() { return voteId; }
    public int getUserId() { return userId; }
    public int getMealChoice() { return mealChoice; }
    public String getDate() { return date; }

    public String toFileString() {
        return voteId + "," + userId + "," + mealChoice + "," + date;
    }

    public static Vote fromFileString(String line) {
        String[] parts = line.split(",");
        return new Vote(
                Integer.parseInt(parts[0]),
                Integer.parseInt(parts[1]),
                Integer.parseInt(parts[2]),
                parts[3]
        );
    }

    @Override
    public String toString() {
        String mealName = switch (mealChoice) {
            case 1 -> "Veg Biryani";
            case 2 -> "Rajma Chawal";
            case 3 -> "Paneer Butter Masala";
            default -> "Unknown";
        };
        return "Vote ID: " + voteId + ", User ID: " + userId + ", Meal: " + mealName + ", Date: " +
date;
    }
}
```

2. dao Package

Class: FeedbackDAO

Purpose: This class handles all file-based CRUD operations for the feedback system.

Main Methods: - addFeedback(Feedback fb) – Appends a new feedback to the file. - List<Feedback> getAllFeedbacks() – Reads all feedback records from the file. - boolean updateFeedback(int id, Feedback updatedFb) – Finds and updates feedback by ID. - boolean deleteFeedback(int id) – Deletes a specific feedback entry by ID.

Note: Uses FileWriter, BufferedReader, FileReader, etc. for file handling.

```
package com.messfeedback.dao;

import com.messfeedback.models.Feedback;
import com.messfeedback.util.FileUtil;

import java.util.ArrayList;
import java.util.List;

public class FeedbackDAO {
   // File name where feedbacks are stored
   private final String FILE_NAME = "feedbacks.txt";

   /**
    * Submit new feedback by appending it to the feedback file.
    * @param feedback Feedback object containing details
    * @return true if feedback was successfully saved, false otherwise
    */
   public boolean submitFeedback(Feedback feedback) {
      try {
         // Prepare CSV line string from feedback object fields
         String line = feedback.getFeedbackId() + "," +
               feedback.getUserId() + "," +
               feedback.getMealType() + "," +
               feedback.getRating() + "," +
               escapeCommas(feedback.getComments()) + "," +  // Handle commas in comments
by replacing them
               feedback.getDate();
```

```
        // Append the feedback line to the file
        FileUtil.appendLine(FILE_NAME, line);
        return true;
    } catch (Exception e) {
        // Print error message if something goes wrong while writing
        System.out.println("Error writing feedback: " + e.getMessage());
        return false;
    }
}

/**
 * Retrieve all feedback entries from the file.
 * @return List of Feedback objects
 */
public List<Feedback> getAllFeedbacks() {
    // Read all lines from the feedback file
    List<String> lines = FileUtil.readFile(FILE_NAME);
    List<Feedback> feedbackList = new ArrayList<>();

    // Process each line to create Feedback objects
    for (String line : lines) {
        // Split line by comma (simple CSV parsing)
        String[] parts = splitCSV(line);

        // Check if line has exactly 6 parts (fields)
        if (parts.length == 6) {
            try {
                // Parse fields from string array
                int feedbackId = Integer.parseInt(parts[0]);
                int userId = Integer.parseInt(parts[1]);
                String mealType = parts[2];
                int rating = Integer.parseInt(parts[3]);
                String comments = parts[4];
                String date = parts[5];

                // Create Feedback object and add to list
                feedbackList.add(new Feedback(feedbackId, userId, mealType, rating, comments,
```

```
date));
            } catch (NumberFormatException e) {
                // Skip invalid lines and notify
                System.out.println("Skipping invalid feedback line: " + line);
            }
        }
    }
    return feedbackList;
}

/**
 * Get feedback list filtered by user ID.
 * @param userId ID of the user
 * @return List of Feedback objects for that user
 */
public List<Feedback> getFeedbacksByUserId(int userId) {
    List<Feedback> all = getAllFeedbacks();
    List<Feedback> result = new ArrayList<>();
    // Filter feedbacks matching the given userId
    for (Feedback f : all) {
        if (f.getUserId() == userId) {
            result.add(f);
        }
    }
    return result;
}

/**
 * Helper method to replace commas in comments with semicolons.
 * This avoids issues with splitting CSV lines by commas.
 * @param input Original comment string
 * @return Comment string with commas replaced
 */
private String escapeCommas(String input) {
    if (input.contains(",")) {
        return input.replace(",", ";"); // simple replacement to avoid CSV split issues
    }
    return input;
```

```
    }

    /**
     * Helper method to split CSV lines by commas.
     * Note: This is a simple split; it doesn't handle quotes or complex CSV cases.
     * @param line CSV line string
     * @return Array of field values
     */
    private String[] splitCSV(String line) {
        return line.split(",");
    }
}
```

Class: UserDAO

```
package com.messfeedback.dao;

import com.messfeedback.models.User;
import com.messfeedback.util.FileUtil;

import java.util.ArrayList;
import java.util.List;

public class UserDAO {
    // File name where user data is stored
    private final String FILE_NAME = "users.txt";

    /**
     * Registers a new user by appending their data to the users file.
     * @param user User object containing user details
     * @return true if registration is successful, false if username is already taken
     */
    public boolean registerUser(User user) {
        // Get all existing users
        List<User> users = getAllUsers();

        // Check for duplicate username
        for (User u : users) {
            if (u.getUsername().equals(user.getUsername())) {
```

```
            return false; // username already exists
        }
    }

    // Prepare line to append to file in CSV format
    String line = user.getUserId() + "," + user.getUsername() + "," + user.getPassword() + "," +
user.getRole();

    // Append the new user line to the file
    FileUtil.appendLine(FILE_NAME, line);
    return true;
}

/**
 * Authenticates a user by matching username and password.
 * @param username Entered username
 * @param password Entered password
 * @return User object if credentials match, null otherwise
 */
public User login(String username, String password) {
    // Read all users from the file
    List<User> users = getAllUsers();

    // Check for username and password match
    for (User user : users) {
        if (user.getUsername().equals(username) && user.getPassword().equals(password)) {
            return user; // successful login
        }
    }

    return null; // no match found
}

/**
 * Reads all user records from the users file.
 * @return List of User objects representing all users
 */
public List<User> getAllUsers() {
```

```
        // Read all lines from file
        List<String> lines = FileUtil.readFile(FILE_NAME);
        List<User> users = new ArrayList<>();

        // Parse each line into a User object
        for (String line : lines) {
            String[] parts = line.split(",");
            if (parts.length == 4) {
                try {
                    int id = Integer.parseInt(parts[0]);
                    String username = parts[1];
                    String password = parts[2];
                    String role = parts[3];

                    users.add(new User(id, username, password, role));
                } catch (NumberFormatException e) {
                    // Skip lines with invalid id
                    System.out.println("Skipping invalid user line: " + line);
                }
            }
        }

        return users;
    }
}
```

Class : VoteDAO

```
// VoteDAO.java
package com.messfeedback.dao;

import com.messfeedback.models.Vote;

import java.io.*;
import java.util.*;

public class VoteDAO {
    private final String FILE_PATH = "votes.txt";
```

```
public boolean saveVote(Vote vote) {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(FILE_PATH, true))) {
        bw.write(vote.toFileString());
        bw.newLine();
        return true;
    } catch (IOException e) {
        return false;
    }
}

public boolean hasUserVotedToday(int userId, String date) {
    List<Vote> votes = getAllVotes();
    for (Vote v : votes) {
        if (v.getUserId() == userId && v.getDate().equals(date)) {
            return true;
        }
    }
    return false;
}

public Map<Integer, Integer> getVoteCounts() {
    List<Vote> votes = getAllVotes();
    Map<Integer, Integer> countMap = new HashMap<>();
    for (Vote v : votes) {
        countMap.put(v.getMealChoice(), countMap.getOrDefault(v.getMealChoice(), 0) + 1);
    }
    return countMap;
}

public List<Vote> getAllVotes() {
    List<Vote> voteList = new ArrayList<>();
    File file = new File(FILE_PATH);
    if (!file.exists()) return voteList;

    try (BufferedReader br = new BufferedReader(new FileReader(FILE_PATH))) {
        String line;
        while ((line = br.readLine()) != null) {
            voteList.add(Vote.fromFileString(line));
```

```
        }
    } catch (IOException e) {
        // Ignore or log
    }
    return voteList;
  }
}
```

3. util Package

Class: FileUtil

Purpose: Provides reusable helper functions for file operations and data conversion.

Key Methods: - int generateId() – Scans the file to return the next available feedback ID. - String toCSV(Feedback fb) – Converts a Feedback object into a CSV line. - Feedback fromCSV(String line) – Converts a CSV line back into a Feedback object.

```
package com.messfeedback.util;

import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class FileUtil {

  // Read all lines from a file
  public static List<String> readFile(String fileName) {
    List<String> lines = new ArrayList<>();
    File file = new File(fileName);

    if (!file.exists()) {
        return lines; // return empty list if file doesn't exist
    }

    try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
        String line;
        while ((line = reader.readLine()) != null) {
            lines.add(line);
```

```
        }
    } catch (IOException e) {
        System.out.println("Error reading file: " + fileName);
    }
    return lines;
  }

  // Write all lines to a file (overwrite)
  public static void writeFile(String fileName, List<String> lines) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName))) {
        for (String line : lines) {
            writer.write(line);
            writer.newLine();
        }
    } catch (IOException e) {
        System.out.println("Error writing to file: " + fileName);
    }
  }

  // Append a line to a file
  public static void appendLine(String fileName, String line) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName, true))) {
        writer.write(line);
        writer.newLine();
    } catch (IOException e) {
        System.out.println("Error appending to file: " + fileName);
    }
  }
}
```

4. ui Package

Class: FeedbackMenu

Purpose: Contains all menu-related logic. Displays console-based menus and takes user input using Scanner.

Main Method: - showMainMenu() – Displays options like Register, Login, Submit/View Feedback, etc., and handles user interaction.

Features: - Displays user-friendly menu. - Takes input from the user. - Calls methods from FeedbackDAO to perform actions. - Validates user inputs.

```
package com.messfeedback.ui;

import com.messfeedback.dao.FeedbackDAO;
import com.messfeedback.dao.UserDAO;
import com.messfeedback.dao.VoteDAO;
import com.messfeedback.models.Feedback;
import com.messfeedback.models.User;
import com.messfeedback.models.Vote;

import java.util.*;

public class FeedbackMenu {
    private Scanner sc = new Scanner(System.in);
    private UserDAO userDAO = new UserDAO();
    private FeedbackDAO feedbackDAO = new FeedbackDAO();
    private VoteDAO voteDAO = new VoteDAO();
    private User currentUser;

    public void start() {
        while (true) {
            System.out.println("\n==== Mess Feedback System ====");
            System.out.println("1. Register");
            System.out.println("2. Login");
            System.out.println("3. Exit");
            System.out.print("Select an option: ");
            int choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {
                case 1 -> register();
                case 2 -> login();
                case 3 -> {
                    System.out.println("Exiting...");
                    return;
                }
```

```
            default -> System.out.println("Invalid option.");
        }
    }
}

private void register() {
    System.out.print("Enter username: ");
    String username = sc.nextLine().trim();

    System.out.print("Enter password: ");
    String password = sc.nextLine().trim();

    System.out.print("Enter role (student/admin): ");
    String role = sc.nextLine().trim();

    // Validation here
    if (username.isEmpty() || password.isEmpty() || role.isEmpty()) {
        System.out.println("Username, password, and role cannot be empty.");
        return;  // method से बाहर निकल जाओ अगर invalid input
    }

    if (!role.equalsIgnoreCase("student") && !role.equalsIgnoreCase("admin")) {
        System.out.println("Invalid role. Please enter 'student' or 'admin'.");
        return;
    }

    User user = new User(0, username, password, role.toLowerCase());
    boolean success = userDAO.registerUser(user);

    if (success) {
        System.out.println("Registered successfully!");
    } else {
        System.out.println("Registration failed. Username might already exist.");
    }
}

private void login() {
    System.out.print("Enter username: ");
```

```
    String username = sc.nextLine().trim();

    System.out.print("Enter password: ");
    String password = sc.nextLine().trim();

    // Validation here
    if (username.isEmpty() || password.isEmpty()) {
        System.out.println("Username and password cannot be empty.");
        return;
    }

    User user = userDAO.login(username, password);

    if (user != null) {
        currentUser = user;
        System.out.println("Login successful!");
        if (user.getRole().equalsIgnoreCase("admin")) {
            adminMenu();
        } else {
            studentMenu();
        }
    } else {
        System.out.println("Invalid credentials.");
    }
}

private void studentMenu() {
    while (true) {
        System.out.println("\nWelcome, " + currentUser.getUsername());
        System.out.println("1. Submit Feedback");
        System.out.println("2. Vote for Meal");
        System.out.println("3. View My Feedback");
        System.out.println("4. Logout");
        System.out.print("Choice: ");
        int choice = sc.nextInt();
        sc.nextLine();

        switch (choice) {
```

```
            case 1 -> submitFeedback();
            case 2 -> voteForMeal();
            case 3 -> viewMyFeedback();
            case 4 -> {
                currentUser = null;
                return;
            }
            default -> System.out.println("Invalid choice.");
        }
    }
}

private void adminMenu() {
    while (true) {
        System.out.println("\nWelcome Admin, " + currentUser.getUsername());
        System.out.println("1. View All Feedback");
        System.out.println("2. View Vote Summary");
        System.out.println("3. Logout");
        System.out.print("Choice: ");
        int choice = sc.nextInt();
        sc.nextLine();

        switch (choice) {
            case 1 -> viewAllFeedback();
            case 2 -> viewVoteSummary();
            case 3 -> {
                currentUser = null;
                return;
            }
            default -> System.out.println("Invalid choice.");
        }
    }
}

private void submitFeedback() {
    System.out.print("Meal Type (Breakfast/Lunch/Dinner): ");
    String meal = sc.nextLine();
```

```
        System.out.print("Rating (1-5): ");
        int rating;
        while (true) {
            System.out.print("Rating (1-5): ");
            if (sc.hasNextInt()) {
                rating = sc.nextInt();
                sc.nextLine();
                if (rating >= 1 && rating <= 5) break;
                else System.out.println("Rating must be between 1 and 5.");
            } else {
                System.out.println("Please enter a valid number.");
                sc.nextLine(); // consume invalid input
            }
        }


        System.out.print("Comments: ");
        String comments = sc.nextLine();

        String date = java.time.LocalDate.now().toString();
        int feedbackId = feedbackDAO.getAllFeedbacks().size() + 1;

        Feedback fb = new Feedback(feedbackId, currentUser.getUserId(), meal, rating, comments,
date);
        boolean success = feedbackDAO.submitFeedback(fb);

        if (success) {
            System.out.println("Feedback submitted successfully.");
        } else {
            System.out.println("Failed to submit feedback.");
        }
    }

    private void voteForMeal() {
        String today = java.time.LocalDate.now().toString();

        if (voteDAO.hasUserVotedToday(currentUser.getUserId(), today)) {
            System.out.println("You have already voted today.");
```

```
        return;
    }

    System.out.println("Today's Meal Options:");
    System.out.println("1. Veg Biryani");
    System.out.println("2. Rajma Chawal");
    System.out.println("3. Paneer Butter Masala");
    System.out.print("Enter your choice (1-3): ");
    int choice = sc.nextInt();
    sc.nextLine();

    if (choice < 1 || choice > 3) {
        System.out.println("Invalid choice.");
        return;
    }

    int voteId = new Random().nextInt(100000);
    Vote vote = new Vote(voteId, currentUser.getUserId(), choice, today);

    if (voteDAO.saveVote(vote)) {
        System.out.println("Vote submitted successfully!");
    } else {
        System.out.println("Failed to submit vote.");
    }
}

private void viewMyFeedback() {
    List<Feedback> list = feedbackDAO.getFeedbacksByUserId(currentUser.getUserId());
    if (list.isEmpty()) {
        System.out.println("No feedback found.");
    } else {
        System.out.println("Your Feedbacks:");
        for (Feedback fb : list) {
            System.out.println(fb);
        }
    }
}
```

```
    private void viewAllFeedback() {
        List<Feedback> list = feedbackDAO.getAllFeedbacks();
        if (list.isEmpty()) {
            System.out.println("No feedback submitted yet.");
        } else {
            System.out.println("All Feedbacks:");
            for (Feedback fb : list) {
                System.out.println(fb);
            }
        }
    }

    private void viewVoteSummary() {
        Map<Integer, Integer> votes = voteDAO.getVoteCounts();
        System.out.println("Voting Summary:");
        System.out.println("1. Veg Biryani: " + votes.getOrDefault(1, 0));
        System.out.println("2. Rajma Chawal: " + votes.getOrDefault(2, 0));
        System.out.println("3. Paneer Butter Masala: " + votes.getOrDefault(3, 0));
    }
}
```

5. Root Package

Class: Main

Purpose: Acts as the entry point of the application.

Main Method: - main(String[] args) – Instantiates FeedbackMenu and calls showMainMenu() to start the system.

```
package com.messfeedback;

import com.messfeedback.ui.FeedbackMenu;

/**
 * Main class to launch the Mess Feedback System application.
 */
public class Main {
    public static void main(String[] args) {
        FeedbackMenu menu = new FeedbackMenu(); // Create instance of the main menu UI
```

```
        menu.start();                    // Start the interactive menu loop
    }
}
```